

citations_agent.py

```
import sys
import os
import requests
import json
from typing import List, Dict, Optional, Tuple, Any
from urllib.parse import urlparse
from dataclasses import dataclass
import re
from concurrent.futures import ThreadPoolExecutor
import logging

current_dir = os.path.dirname(os.path.abspath(__file__))
parent_dir = os.path.dirname(current_dir)
project_root = os.path.dirname(parent_dir)

sys.path.append(parent_dir)
sys.path.append(project_root)

from agno.agent import Agent
from agno.models.google import Gemini
from dotenv import load_dotenv
from agno.tools.tavily import TavilyTools
from Tools.web_scrapper import scrape_agri_prices, scrape_policy_updates, scrape_links

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
load_dotenv()

@dataclass
class Citation:
    title: str
    authors: str
    journal: str
    year: str
    url: str
    source_type: str
    relevance: int
    doi: Optional[str] = None
    abstract: Optional[str] = None
    keywords: List[str] = None

    def to_apo(self) -> str:
        return f"{self.authors} ({self.year}). {self.title}. {self.journal}."

class EnhancedCitationAgent:
    def __init__(self):
        self.agent = Agent(
            model=Gemini(id="gemini-2.0-flash"),
            tools=[TavilyTools(), scrape_agri_prices, scrape_policy_updates, scrape_links],
            instructions=self._get_enhanced_instructions(),
            markdown=True,
        )

        self.trusted_domains = [
            'nature.com', 'sciencedirect.com', 'springer.com', 'wiley.com',
            'tandfonline.com', 'cambridge.org', 'oxford.com', 'plos.org',
            'usda.gov', 'fao.org', 'who.int', 'cgia.org', 'worldbank.org',
            'nih.gov', 'pubmed.ncbi.nlm.nih.gov', 'researchgate.net',
            'frontiersin.org', 'mdpi.com', 'elsevier.com', 'jstor.org',
            'journals.ashs.org', 'link.springer.com', 'journals.plos.org',
            'onlinelibrary.wiley.com', 'academic.oup.com', 'emerald.com'
        ]

        self.source_quality_scores = {
            'journal_article': 10,
            'conference_paper': 8,
            'government_report': 9,
            'university_publication': 8,
            'extension_publication': 7,
            'industry_report': 6,
            'news_article': 4,
            'blog_post': 3
        }
```

```

def _get_enhanced_instructions(self) -> str:
    return """
    You are an advanced agricultural research citation specialist with enhanced capabilities.

    SEARCH STRATEGY:
    1. Use TavilyTools for comprehensive academic and research searches
    2. Use scrape_links to extract citations from institutional research pages
    3. Use scrape_policy_updates for government agricultural research and policies
    4. Use scrape_agri_prices for agricultural economic research and market studies

    SEARCH EXECUTION:
    - Execute multiple targeted searches using different keywords and phrases
    - Search academic journals, government databases, and institutional repositories
    - Focus on peer-reviewed sources with DOIs when possible
    - Prioritize recent publications (last 5 years) unless specified otherwise
    - Cross-reference findings across multiple sources

    CITATION EXTRACTION:
    - Extract complete bibliographic information
    - Identify and validate DOIs and URLs
    - Capture abstracts when available
    - Determine source quality and type
    - Rate relevance to the research topic (1-10 scale)

    OUTPUT FORMAT:
    For each citation found, provide:
    - Title: [Complete title]
    - Authors: [Full author list]
    - Journal: [Publication venue]
    - Year: [Publication year]
    - URL: [Direct link or DOI]
    - Type: [Source type]
    - Relevance: [1-10 score]
    - Abstract: [If available]

    QUALITY STANDARDS:
    - Only include citations from reputable sources
    - Verify all URLs are accessible
    - Ensure direct relevance to agricultural research
    - Prioritize peer-reviewed and authoritative sources
    """

def _validate_url_batch(self, urls: List[str]) -> Dict[str, bool]:
    def check_url(url: str) -> Tuple[str, bool]:
        try:
            if not url or not url.startswith(('http://', 'https://')):
                return url, False

            parsed_url = urlparse(url)
            domain = parsed_url.netloc.lower().replace('www.', '')

            is_trusted = any(trusted in domain for trusted in self.trusted_domains)
            if not is_trusted:
                return url, False

            response = requests.head(url, timeout=10, allow_redirects=True)
            return url, response.status_code == 200

        except Exception:
            return url, False

    with ThreadPoolExecutor(max_workers=5) as executor:
        results = list(executor.map(check_url, urls))

    return dict(results)

def _extract_citations_enhanced(self, response_text: str) -> List[Citation]:
    citations = []

    citation_patterns = [
        r'Title:\s*(.+?)(?:\n|$)',
        r'\*\Title\*\*\s*(.+?)(?:\n|$)',
        r'# (.+?)(?:\n|$)'
    ]

    author_patterns = [
        r'Authors?:\s*(.+?)(?:\n|$)',

```

```

        r'\*\*Authors?\*\*:s*(.+?)(?:\n|$)'
    ]

    journal_patterns = [
        r'Journal:s*(.+?)(?:\n|$)',
        r'Source:s*(.+?)(?:\n|$)',
        r'\*\*Journal\*\*:s*(.+?)(?:\n|$)'
    ]

    year_patterns = [
        r'Year:s*(\d{4})',
        r'\*\*Year\*\*:s*(\d{4})',
        r'\((\d{4})\)'
    ]

    url_patterns = [
        r'URL:s*(https?://[^\s\n]+)',
        r'DOI:s*(https?://[^\s\n]+)',
        r'Link:s*(https?://[^\s\n]+)',
        r'(https?://[^\s\n]+)'
    ]

    sections = re.split(r'\n\s*\n', response_text)

    for section in sections:
        if len(section.strip()) < 50:
            continue

        citation_data = {}

        for pattern in citation_patterns:
            match = re.search(pattern, section)
            if match:
                citation_data['title'] = match.group(1).strip()
                break

        for pattern in author_patterns:
            match = re.search(pattern, section)
            if match:
                citation_data['authors'] = match.group(1).strip()
                break

        for pattern in journal_patterns:
            match = re.search(pattern, section)
            if match:
                citation_data['journal'] = match.group(1).strip()
                break

        for pattern in year_patterns:
            match = re.search(pattern, section)
            if match:
                citation_data['year'] = match.group(1).strip()
                break

        urls_found = []
        for pattern in url_patterns:
            matches = re.findall(pattern, section)
            urls_found.extend(matches)

        if urls_found:
            citation_data['url'] = urls_found[0]

        relevance_match = re.search(r'Relevance:s*(\d+)', section)
        relevance = int(relevance_match.group(1)) if relevance_match else 7

        type_match = re.search(r'Type:s*(.+?)(?:\n|$)', section)
        source_type = type_match.group(1).strip() if type_match else 'journal_article'

        abstract_match = re.search(r'Abstract:s*(.+?)(?:\n\n|$)', section, re.DOTALL)
        abstract = abstract_match.group(1).strip() if abstract_match else None

        if 'title' in citation_data and len(citation_data) >= 3:
            citation = Citation(
                title=citation_data.get('title', ''),
                authors=citation_data.get('authors', 'Unknown'),
                journal=citation_data.get('journal', 'Unknown'),
                year=citation_data.get('year', 'Unknown'),

```

```

        url=citation_data.get('url', 'Unknown'),
        abstract=abstract,
        relevance=relevance,
        source_type=source_type
    )
    citations.append(citation)

return citations

def find_citations_basic(self, topic: str, num_citations: int = 15,
                        focus_areas: List[str] = None) -> Dict[str, Any]:
    """Basic citation finding method"""
    logger.info(f"Starting basic citation search for: {topic}")

    try:
        search_query = f"""
        Find academic citations and research papers for: {topic}

        Search requirements:
        - Find {num_citations} high-quality academic citations
        - Focus on peer-reviewed journal articles
        - Include government reports and institutional publications
        - Prioritize recent publications (last 5 years)
        - Extract complete bibliographic information
        """

        if focus_areas:
            search_query += f"\nSpecial focus on: {' '.join(focus_areas)}"

        response = self.agent.run(search_query)

        response_text = response.content if hasattr(response, 'content') else str(response)
        citations = self._extract_citations_enhanced(response_text)

        valid_citations = []
        urls_to_check = []

        for citation in citations:
            if citation.url and citation.url != 'Unknown':
                urls_to_check.append(citation.url)

        url_validity = {}
        if urls_to_check:
            url_validity = self._validate_url_batch(urls_to_check)

        for citation in citations:
            if citation.url == 'Unknown' or url_validity.get(citation.url, True):
                valid_citations.append(citation)

        valid_citations.sort(key=lambda c: c.relevance, reverse=True)
        valid_citations = valid_citations[:num_citations]

        logger.info(f"Found {len(valid_citations)} valid citations")

        return {
            "success": len(valid_citations) > 0,
            "citations": valid_citations,
            "valid_count": len(valid_citations),
            "total_found": len(citations),
            "search_query": search_query,
            "focus_areas": focus_areas
        }
    except Exception as e:
        logger.error(f"Error during basic citation search: {str(e)}")
        return {
            "success": False,
            "error": str(e),
            "citations": [],
            "valid_count": 0,
            "total_found": 0
        }

def find_citations_enhanced(self, topic: str, num_citations: int = 15,
                            focus_areas: List[str] = None) -> Dict[str, Any]:
    """Enhanced citation finding with comprehensive search across multiple sources"""
    logger.info(f"Starting enhanced citation search for: {topic}")
    try:

```

```

search_query = f"""
Find academic citations and research papers for: {topic}

Search requirements:
- Find {num_citations} high-quality academic citations
- Focus on peer-reviewed journal articles
- Include government reports and institutional publications
- Prioritize recent publications (last 5 years)
- Extract complete bibliographic information
"""

if focus_areas:
    search_query += f"\nSpecial focus on: {' ', ' '.join(focus_areas)}"

response = self.agent.run(search_query)

response_text = response.content if hasattr(response, 'content') else str(response)
citations = self._extract_citations_enhanced(response_text)

valid_citations = []
urls_to_check = []

for citation in citations:
    if citation.url and citation.url != 'Unknown':
        urls_to_check.append(citation.url)

url_validity = {}
if urls_to_check:
    url_validity = self._validate_url_batch(urls_to_check)

for citation in citations:
    if citation.url == 'Unknown' or url_validity.get(citation.url, True):
        valid_citations.append(citation)

valid_citations.sort(key=lambda c: c.relevance, reverse=True)
valid_citations = valid_citations[:num_citations]

logger.info(f"Found {len(valid_citations)} valid citations")

return {
    "success": len(valid_citations) > 0,
    "citations": valid_citations,
    "valid_count": len(valid_citations),
    "total_found": len(citations),
    "search_query": search_query,
    "focus_areas": focus_areas
}
except Exception as e:
    logger.error(f"Error during enhanced citation search: {str(e)}")
    return {
        "success": False,
        "error": str(e),
        "citations": [],
        "valid_count": 0,
        "total_found": 0
    }
}

if __name__ == "__main__":
    agent = EnhancedCitationAgent()

    result = agent.find_citations_basic(
        topic="Soil Health Improvement in Wheat Farming",
        num_citations=5,
        focus_areas=["sustainability", "fertilizer management"]
    )

    if result["success"]:
        print(f"\n■ Found {result['valid_count']} valid citations (out of {result['total_found']})\n")
        for i, citation in enumerate(result["citations"], 1):
            print(f"{i}. {citation.to_apa()}")
            print(f"    URL: {citation.url}")
            print(f"    Relevance: {citation.relevance}/10\n")
    else:
        print("\n■ No citations found.")
        if "error" in result:
            print(f"Error: {result['error']}")

```

subsearchAgent.py

```
import os
import sys
import asyncio
import threading
from concurrent.futures import ThreadPoolExecutor, as_completed
from typing import Dict, List, Any, Optional, Tuple
from dataclasses import dataclass
from datetime import datetime
import logging

current_dir = os.path.dirname(os.path.abspath(__file__))
parent_dir = os.path.dirname(current_dir)
sys.path.append(parent_dir)

from agno.agent import Agent
from agno.models.google import Gemini
from agno.tools.arxiv import ArxivTools
from agno.tools.wikipedia import WikipediaTools
from agno.tools.tavily import TavilyTools
from Tools.fetchWeatherForecast import get_google_weather_forecast
from Tools.getCropRecommendation import get_crop_recommendation
from Tools.pest_prediction import detect_pests
from Tools.risk_management import get_agricultural_risk_metrics
from dotenv import load_dotenv

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
load_dotenv()

@dataclass
class SearchResult:
    agent_name: str
    query: str
    content: str
    success: bool
    execution_time: float
    sources_found: int = 0
    error_message: Optional[str] = None

class EnhancedSubsearchAgent:
    def __init__(self, max_workers: int = 3):
        self.max_workers = max_workers
        self.agents = self._initialize_agents()

    def _initialize_agents(self) -> Dict[str, Agent]:
        common_tools = [
            get_google_weather_forecast,
            get_crop_recommendation,
            detect_pests,
            get_agricultural_risk_metrics
        ]

        agents = {
            "arxiv": Agent(
                name="ArXiv Research Specialist",
                role="Academic research and scientific papers",
                model=Gemini(id="gemini-2.0-flash"),
                tools=[ArxivTools()] + common_tools,
                instructions="""
                You are the ArXiv Research Specialist focusing on academic agricultural research.
                - Search and analyze recent academic papers
                - Provide comprehensive literature reviews
                - Focus on peer-reviewed research and citations
                - Extract key findings and methodologies
                - Identify research gaps and trends
                Complete your analysis independently with detailed academic insights.
                """
            ),
            "wikipedia": Agent(
```

```

        name="Knowledge Base Specialist",
        role="Foundational knowledge and context",
        model=Gemini(id="gemini-2.0-flash"),
        tools=[WikipediaTools()] + common_tools,
        instructions="""
        You are the Knowledge Base Specialist providing foundational agricultural knowledge.
        - Provide comprehensive background information
        - Explain fundamental concepts and definitions
        - Give historical context and established facts
        - Cross-reference information for accuracy
        - Build knowledge foundation for complex topics
        Complete your research independently with authoritative sources.
        """
    ),

    "tavily": Agent(
        name="Current Intelligence Specialist",
        role="Real-time information and market intelligence",
        model=Gemini(id="gemini-2.0-flash"),
        tools=[TavilyTools()] + common_tools,
        instructions="""
        You are the Current Intelligence Specialist for real-time agricultural information.
        - Search current web developments and news
        - Provide market intelligence and trends
        - Find recent policy changes and regulations
        - Gather real-time agricultural data and statistics
        - Monitor emerging technologies and practices
        Complete your analysis independently with current, actionable insights.
        """
    )
}

return agents

def _execute_single_search(self, agent_name: str, query: str, context: Optional[str] = None) -> SearchResult:
    start_time = datetime.now()
    agent = self.agents[agent_name]

    try:
        research_query = self._prepare_query(query, context, agent_name)
        response = agent.run(research_query)

        content = response.content if hasattr(response, 'content') else str(response)
        execution_time = (datetime.now() - start_time).total_seconds()

        sources_count = self._count_sources(content)

        return SearchResult(
            agent_name=agent_name,
            query=query,
            content=content,
            success=True,
            execution_time=execution_time,
            sources_found=sources_count
        )

    except Exception as e:
        execution_time = (datetime.now() - start_time).total_seconds()
        logger.error(f"Search failed for {agent_name}: {str(e)}")

        return SearchResult(
            agent_name=agent_name,
            query=query,
            content="",
            success=False,
            execution_time=execution_time,
            error_message=str(e)
        )

def _prepare_query(self, query: str, context: Optional[str], agent_name: str) -> str:
    base_query = f"Agricultural Research Query: {query}\n\n"

    if context:
        base_query += f"Context: {context}\n\n"

    agent_specific_instructions = {
        "arxiv": "Focus on academic papers, research methodologies, and peer-reviewed findings."
    }

```

```

        "wikipedia": "Provide foundational knowledge, definitions, and established facts.",
        "tavily": "Find current developments, market trends, and real-time data."
    }

    base_query += f"Instructions for {agent_name}:\n"
    base_query += agent_specific_instructions.get(agent_name, "Provide comprehensive analysis.")
    base_query += "\n\nProvide detailed analysis with sources, key findings, and actionable insights."

    return base_query

def _count_sources(self, content: str) -> int:
    source_indicators = ['http', 'doi:', 'source:', 'reference:', 'cited', 'published']
    return sum(1 for indicator in source_indicators if indicator.lower() in content.lower())

def subsearch_single_query(self, query: str, context: Optional[str] = None) -> Dict[str, Any]:
    """
    Execute a single query across all available agents and return combined results.
    This method is required by the workflow.py file.
    """
    start_time = datetime.now()
    results = []

    # Execute search across all agents for this single query
    for agent_name in self.agents.keys():
        result = self._execute_single_search(agent_name, query, context)
        results.append(result)

        if result.success:
            logger.info(f"Single query search - {agent_name}: Found {result.sources_found} sources")
        else:
            logger.warning(f"Single query search - {agent_name}: Search failed - {result.error_message}")

    total_time = (datetime.now() - start_time).total_seconds()
    successful_results = [r for r in results if r.success]
    failed_results = [r for r in results if not r.success]

    # Combine all successful results into a single content block
    combined_content = self._combine_single_query_results(successful_results, query)

    # Create sources list from successful results
    sources = []
    for result in successful_results:
        # Extract potential source information from content
        source_info = {
            "agent": result.agent_name,
            "query": result.query,
            "sources_found": result.sources_found,
            "execution_time": result.execution_time,
            "content_preview": result.content[:200] + "..." if len(result.content) > 200 else result.content
        }
        sources.append(source_info)

    return {
        "success": len(successful_results) > 0,
        "query": query,
        "combined_content": combined_content,
        "sources": sources,
        "sources_found": len(sources),
        "successful_searches": len(successful_results),
        "failed_searches": len(failed_results),
        "total_sources_found": sum(r.sources_found for r in successful_results),
        "execution_time": total_time,
        "agents_used": [r.agent_name for r in successful_results],
        "errors": [{"agent": r.agent_name, "error": r.error_message} for r in failed_results] if failed_results
    }

def _combine_single_query_results(self, results: List[SearchResult], original_query: str) -> str:
    """Combine results from multiple agents for a single query"""
    if not results:
        return f"No successful results found for query: {original_query}"

    combined = f"# Research Results for: {original_query}\n\n"
    combined += f"*Compiled from {len(results)} successful agent searches*\n\n"

    # Group results by agent for better organization
    for i, result in enumerate(results, 1):
        combined += f"## {result.agent_name} Analysis\n"

```



```

        combined += f"**Sources Found:** {result.sources_found} | **Search Time:** {result.execution_time}\n"
        combined += f"{result.content}\n\n"

        if i < len(results):
            combined += "---\n\n"

    # Add summary section
    total_sources = sum(r.sources_found for r in results)
    combined += f"\n## Search Summary\n"
    combined += f"- **Total Sources Found:** {total_sources}\n"
    combined += f"- **Agents Consulted:** {' '.join([r.agent_name for r in results])}\n"
    combined += f"- **Query Processed:** {original_query}\n"

    return combined

def search_parallel(self, queries: List[str], context: Optional[str] = None) -> List[SearchResult]:
    all_results = []

    with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
        futures = []

        for query in queries:
            for agent_name in self.agents.keys():
                future = executor.submit(self._execute_single_search, agent_name, query, context)
                futures.append(future)

        for future in as_completed(futures):
            try:
                result = future.result(timeout=120)
                all_results.append(result)
                logger.info(f"Completed search: {result.agent_name} - {result.query[:50]}...")
            except Exception as e:
                logger.error(f"Future execution failed: {str(e)}")

    return all_results

def search_sequential(self, queries: List[str], context: Optional[str] = None) -> List[SearchResult]:
    all_results = []

    for query in queries:
        logger.info(f"Processing query: {query}")

        for agent_name in self.agents.keys():
            result = self._execute_single_search(agent_name, query, context)
            all_results.append(result)

            if result.success:
                logger.info(f"Success {agent_name}: Found {result.sources_found} sources")
            else:
                logger.warning(f"Failed {agent_name}: Search failed")

    return all_results

def search_optimized(self, queries: List[str], context: Optional[str] = None,
                    parallel: bool = True) -> Dict[str, Any]:
    start_time = datetime.now()

    if parallel and len(queries) > 1:
        results = self.search_parallel(queries, context)
        approach = "parallel_execution"
    else:
        results = self.search_sequential(queries, context)
        approach = "sequential_execution"

    total_time = (datetime.now() - start_time).total_seconds()

    successful_results = [r for r in results if r.success]
    failed_results = [r for r in results if not r.success]

    combined_content = self._combine_results(successful_results)

    return {
        "success": len(successful_results) > 0,
        "approach": approach,
        "total_execution_time": total_time,
        "queries_processed": len(queries),
        "successful_searches": len(successful_results),
    }

```

```

        "failed_searches": len(failed_results),
        "total_sources_found": sum(r.sources_found for r in successful_results),
        "combined_content": combined_content,
        "detailed_results": successful_results,
        "errors": [{"agent": r.agent_name, "query": r.query, "error": r.error_message}
                    for r in failed_results],
        "performance_metrics": self._calculate_performance_metrics(results)
    }

def _combine_results(self, results: List[SearchResult]) -> str:
    if not results:
        return "No successful search results found."

    combined = f"## Comprehensive Agricultural Research Results\n"
    combined += f"*Generated from {len(results)} successful searches*\n\n"

    by_agent = {}
    for result in results:
        if result.agent_name not in by_agent:
            by_agent[result.agent_name] = []
        by_agent[result.agent_name].append(result)

    for agent_name, agent_results in by_agent.items():
        combined += f"## {agent_name.title()} Research Findings\n\n"

        for result in agent_results:
            combined += f"### Query: {result.query}\n"
            combined += f"***Sources Found:** {result.sources_found} | **Time:** {result.execution_time}\n"
            combined += f"{result.content}\n\n"
            combined += "---\n\n"

    return combined

def _calculate_performance_metrics(self, results: List[SearchResult]) -> Dict[str, float]:
    if not results:
        return {}

    successful = [r for r in results if r.success]

    return {
        "success_rate": len(successful) / len(results) * 100,
        "average_execution_time": sum(r.execution_time for r in results) / len(results),
        "average_sources_per_search": sum(r.sources_found for r in successful) / len(successful),
        "fastest_search": min(r.execution_time for r in results),
        "slowest_search": max(r.execution_time for r in results)
    }

def execute_parallel_research(self, research_queries: List[str], context: Optional[str] = None) -> Dict[str, Any]:
    """Execute parallel research across multiple queries and return comprehensive results"""
    return self.search_optimized(research_queries, context, parallel=True)

def search(self, queries: List[str], context: Optional[str] = None, parallel: bool = True) -> Dict[str, Any]:
    return self.search_optimized(queries, context, parallel)

if __name__ == "__main__":
    agent = EnhancedSubsearchAgent()

    query = "Soil Health Improvement in Wheat Farming"
    context = "Focus on sustainability, fertilizer management, and crop yield optimization."

    result = agent.subsearch_single_query(query, context)

    print("\n■ Subsearch Agent Results\n")
    if result["success"]:
        print(result["combined_content"])
        print("\n■ Sources Summary:")
        for src in result["sources"]:
            print(f"- Agent: {src['agent']} | Sources Found: {src['sources_found']} | Time: {src['execution_time']} | Content: {src['content_preview']}")
    else:
        print("■ No successful results.")
        if result["errors"]:
            for err in result["errors"]:
                print(f"Agent {err['agent']} failed with error: {err['error']}")

```